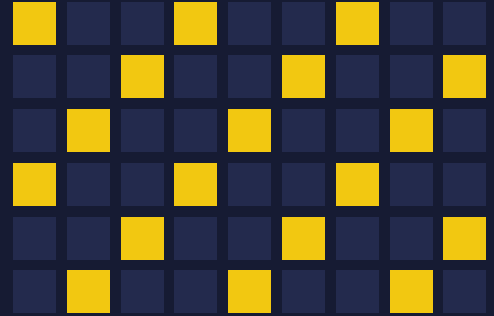




POWER BI NOTES

Basic to Advanced — A Practical Guide for Data Analysts

ADITYA KALYAN | DATA DETECTIVE CLUB

Table of Contents

1. Power BI Fundamentals
2. Connecting to Data & Power Query (M)
3. Data Modeling & Relationships
4. DAX Basics — Columns, Measures & Core Functions
5. DAX Intermediate — CALCULATE, Filter Context & Time Intelligence
6. DAX Advanced — Variables, Iterators & Patterns
7. Visualizations & Report Design
8. Performance Optimization & Best Practices
9. Publishing, Sharing & Row-Level Security
10. Quick-Reference Cheat Sheet

1. Power BI Fundamentals

What Power BI Is

Power BI is Microsoft's business intelligence platform for connecting to data, modeling it, and turning it into interactive reports and dashboards. It has three main pieces:

- **Power BI Desktop** — free Windows app where you build data models, write DAX, and design reports.
- **Power BI Service** (app.powerbi.com) — cloud platform for publishing, sharing, and scheduling refreshes.
- **Power BI Mobile** — apps for viewing reports and getting alerts on the go.

The Three Views in Desktop

- **Report View** — where you build visuals, pages, and the reader-facing layout.
- **Data View** — a grid view of every table currently loaded into the model.
- **Model View** — the diagram of tables and the relationships between them.

Typical Analyst Workflow

- Get Data → connect to a source (Excel, SQL Server, SharePoint, web API, etc.).
- Transform in Power Query → clean, reshape, and load only what you need.
- Model → build relationships between fact and dimension tables.
- Measure → write DAX to calculate KPIs.
- Visualize → build report pages with charts, slicers, and filters.
- Publish → push to the Service and share with a workspace or app.

Analyst tip: The 80/20 rule of Power BI — most of your time should go into Power Query and modeling, not visuals. A clean model makes every visual and DAX formula simpler later.

2. Connecting to Data & Power Query (M)

Common Data Sources

- Files: Excel, CSV, JSON, PDF, folders of files.
- Databases: SQL Server, PostgreSQL, MySQL, Oracle.
- Cloud/Online: SharePoint, Dataverse, Azure services, Web/OData APIs.

Power Query Editor — Core Transformations

Power Query uses a functional language called **M** behind every point-and-click step. Every transformation you apply is recorded as a step, so the whole process replays automatically the next time you hit Refresh.

Transformation	What it does	Where to find it
Change Type	Sets a column's data type (text, whole number, date, etc.)	Column header icon
Remove Columns	Drops columns you don't need before loading	Home > Remove Columns
Split Column	Breaks one column into many by delimiter or position	Transform > Split Column
Merge Queries	Joins two queries like a SQL JOIN (left, inner, anti, etc.)	Home > Merge Queries
Append Queries	Stacks two queries with the same columns (like UNION)	Home > Append Queries
Unpivot Columns	Turns wide data (one column per month) into long/tidy data	Transform > Unpivot
Group By	Aggregates rows, e.g. total sales per region	Transform > Group By

Example: A Simple M Query

```
let
    Source = Excel.Workbook(File.Contents("Sales.xlsx"), null, true),
    Sales_Table = Source[[Item="Sales",Kind="Table"]][Data],
    ChangedType = Table.TransformColumnTypes(Sales_Table,
        {"OrderDate", type date}, {"Amount", type number}),
    Filtered = Table.SelectRows(ChangedType, each [Amount] > 0)
in
    Filtered
```

This loads a Sales table, fixes column types, then keeps only positive amounts.

Query folding: when your source is a database, Power Query tries to push your steps back into SQL so filtering happens on the server, not after every row is pulled into memory. Keep folding-breaking steps (like custom columns) near the end of your query.

3. Data Modeling & Relationships

Star Schema — The Gold Standard

A star schema separates data into **fact tables** (transactions — sales, orders, clicks) and **dimension tables** (descriptive context — Date, Product, Customer, Region). Facts are long and narrow; dimensions are short and wide.

- **Fact table:** one row per event, mostly numeric measures and foreign keys.
- **Dimension table:** one row per entity, full of descriptive text attributes.
- Avoid snowflaking (splitting dimensions into sub-dimensions) unless you have a real reason — it adds joins for no benefit in Power BI.

Relationships

Property	Typical setting	Why
Cardinality	One (dimension) to Many (fact)	Matches how real business data is structured
Cross-filter direction	Single, from dimension to fact	Predictable filtering; avoids ambiguous results
Active/Inactive	One active relationship per pair of tables	Use USERELATIONSHIP() in DAX to activate an inactive one when needed

Example: Activating an Inactive Relationship

```
Shipped Sales =  
CALCULATE (  
    SUM ( Sales[Amount] ),  
    USERELATIONSHIP ( Sales[ShipDate], 'Date'[Date] )  
)
```

Use this when a fact table has two date columns (OrderDate, ShipDate) linked to one Date table.

Building a Date Table

Almost every model needs a dedicated Date table for time intelligence to work correctly.

```
Date =  
ADDCOLUMNS (   
    CALENDAR ( DATE(2022,1,1), DATE(2026,12,31) ),  
    "Year", YEAR ( [Date] ),  
    "Month", FORMAT ( [Date], "MMM" ),  
    "MonthNumber", MONTH ( [Date] ),  
    "Quarter", "Q" & FORMAT ( [Date], "Q" )  
)
```

4. DAX Basics — Columns, Measures & Core Functions

Calculated Column vs Measure

	Calculated Column	Measure
Computed	Row by row, stored in the model	On the fly, at query time
Uses memory?	Yes — takes up space like a real column	No — computed only when displayed
Responds to filters/slicers?	No, fixed once refreshed	Yes, always recalculates
Best for	Categorizing rows (e.g. Price Tier)	Aggregating and summarizing (e.g. Total Sales)

Core Aggregation Functions

```
Total Sales      = SUM ( Sales[Amount] )
Average Order    = AVERAGE ( Sales[Amount] )
Order Count      = COUNTROWS ( Sales )
Distinct Customers = DISTINCTCOUNT ( Sales[CustomerID] )
Safe Margin %    = DIVIDE ( [Total Profit], [Total Sales], 0 )
```

DIVIDE() is preferred over the / operator because it handles divide-by-zero cleanly.

Basic Logic

```
Price Tier =
IF ( Products[UnitPrice] > 500, "Premium",
    IF ( Products[UnitPrice] > 100, "Mid", "Budget" ) )
```

Analyst tip: Write measures, not calculated columns, whenever the result needs to change based on what the user has filtered or sliced — that's 90% of the KPIs you'll build.

5. DAX Intermediate — CALCULATE, Filter Context & Time Intelligence

CALCULATE — the Most Important Function in DAX

CALCULATE evaluates an expression in a modified filter context. Every advanced DAX pattern is really just a creative use of CALCULATE.

```
Sales - Electronics =  
CALCULATE (  
    [Total Sales],  
    Products[Category] = "Electronics"  
)
```

Removing Filters with ALL

```
% of Total Sales =  
DIVIDE (  
    [Total Sales],  
    CALCULATE ( [Total Sales], ALL ( Sales ) )  
)
```

ALL() removes filters so the denominator always sees the grand total, regardless of slicers.

Time Intelligence

Function	Purpose
TOTALYTD / TOTALQTD / TOTALMTD	Running total from the start of the year/quarter/month
SAMEPERIODLASTYEAR	Shifts the date filter back exactly one year
DATEADD	Shifts dates by any interval (days, months, years)
PARALLELPERIOD	Like DATEADD but returns full periods, not partial ones

```
Sales YTD =  
TOTALYTD ( [Total Sales], 'Date'[Date] )  
  
Sales LY =  
CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )  
  
YoY Growth % =  
DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )
```

Filter context vs row context: visuals and slicers create *filter context*. Calculated columns and iterators (the X functions) additionally create *row context*. Understanding which context you're in explains almost every DAX bug.

6. DAX Advanced — Variables, Iterators & Patterns

Variables (VAR / RETURN)

Variables make formulas easier to read and faster to run, since a variable is computed only once even if referenced multiple times.

```
Profit Margin % =  
VAR TotalRevenue = [Total Sales]  
VAR TotalCost = SUM ( Sales[Cost] )  
VAR Margin = TotalRevenue - TotalCost  
RETURN  
    DIVIDE ( Margin, TotalRevenue )
```

Iterators — the X Functions

SUMX, AVERAGEX, and RANKX evaluate an expression row-by-row before aggregating — essential when the calculation itself needs multiplication across rows (e.g. Quantity × Price).

```
Total Revenue (row-level) =  
SUMX ( Sales, Sales[Quantity] * Sales[UnitPrice] )  
  
Customer Rank by Sales =  
RANKX ( ALL ( Customer[CustomerName] ), [Total Sales] )
```

Pattern: Running Total

```
Running Total =  
CALCULATE (  
    [Total Sales],  
    FILTER (  
        ALL ( 'Date'[Date] ),  
        'Date'[Date] <= MAX ( 'Date'[Date] )  
    )  
)
```

Pattern: ABC / Pareto Classification

```
Cumulative % =  
VAR CurrentRank = [Customer Rank by Sales]  
VAR CumulativeSales =  
    CALCULATE (  
        [Total Sales],  
        FILTER ( ALL ( Customer ), [Customer Rank by Sales] <= CurrentRank )  
    )  
RETURN  
    DIVIDE ( CumulativeSales, CALCULATE ( [Total Sales], ALL ( Customer ) ) )
```

Combine with a slicer to flag the top 20% of customers driving 80% of revenue.

7. Visualizations & Report Design

Choosing the Right Visual

Goal	Recommended visual
Compare categories	Bar / column chart
Show a trend over time	Line chart
Show part-to-whole	Stacked bar or 100% stacked bar (avoid pie for >5 slices)
Show a single KPI	Card or KPI visual with a target
Show correlation	Scatter chart
Let users explore detail	Table or matrix with drill-down
Show geography	Map or filled map

Interactivity Tools

- **Slicers** — visible filters users can click; sync them across pages via the Sync Slicers pane.
- **Drillthrough** — right-click a data point to jump to a detail page filtered to that context.
- **Bookmarks** — save a snapshot of filters/visibility to build guided narratives or toggle views.
- **Tooltips pages** — a mini report page shown on hover for extra context.
- **Cross-filtering/highlighting** — clicking one visual filters or highlights the others on the page.

Conditional Formatting

Apply conditional formatting to table/matrix cells or card values (color scales, data bars, icons, or a rules-based formula) to surface outliers without adding another visual.

Design tip: Keep 5–7 visuals per page maximum, align them to a grid, and put the single most important number top-left — that's where the eye lands first.

8. Performance Optimization & Best Practices

Modeling Best Practices

- Prefer a star schema — avoid many-to-many relationships unless unavoidable.
- Set cross-filter direction to Single wherever possible; Both can cause ambiguous, slow queries.
- Remove unused columns and rows in Power Query before loading — smaller model, faster refresh.
- Use whole numbers and reduce high-cardinality text columns (e.g. don't import a free-text Notes column).
- Disable auto date/time hierarchies if you have your own Date table (File > Options > Data Load).

DAX Performance

- Use variables to avoid recalculating the same expression twice.
- Use DIVIDE() instead of the / operator to avoid errors and improve plan quality.
- Filter with column predicates (e.g. Sales[Year] = 2026) instead of full-table FILTER() when possible — it lets the engine use its fastest filtering path.
- Avoid calculated columns for anything that can be a measure — they bloat model size.

Refresh & Scale

- Use Incremental Refresh for large fact tables — only new/changed partitions are reprocessed.
- Use Aggregations for very large models so visuals hit a small pre-summarized table first.
- Check performance with Performance Analyzer (View tab) to see which visual or DAX query is slow.

9. Publishing, Sharing & Row-Level Security

Power BI Service Essentials

- **Workspace** — a container for datasets, reports, and dashboards shared with a team.
- **App** — a packaged, read-only bundle of reports distributed to a wider audience.
- **Dataset** — the published model; can be reused across multiple reports.
- **Scheduled refresh** — configure how often the Service pulls fresh data (requires a gateway for on-prem sources).

Row-Level Security (RLS)

RLS restricts what rows each user sees, based on their identity, using a DAX filter expression on a role.

```
[Region] = LOOKUPVALUE (
    UserRegion[Region],
    UserRegion[Email], USERPRINCIPALNAME()
)
```

Defined inside Modeling > Manage Roles, then tested with 'View As Roles'.

Static vs Dynamic RLS: static RLS hardcodes a value per role (e.g. Region = "West"); dynamic RLS (shown above) looks up the right filter per logged-in user from a mapping table — the scalable option for larger teams.

10. Quick-Reference Cheat Sheet

Category	Function / Concept	Use it for
Aggregation	SUM, AVERAGE, COUNTROWS, DIVIDE	Basic totals and safe division
Filter	CALCULATE, FILTER, ALL, ALLEXCEPT	Changing or removing filter context
Time Intelligence	TOTALYTD, SAMEPERIODLASTYEAR, DATEADD	Period-over-period comparisons
Iterators	SUMX, AVERAGEX, RANKX	Row-by-row math before aggregating
Logic	IF, SWITCH, AND, OR	Branching / categorization logic
Text	CONCATENATE, FORMAT, LOOKUPVALUE	Building labels and lookups
Modeling	Star schema, Single-direction filters	A model that's fast and predictable
Performance	Variables, Incremental Refresh, Aggregations	Keeping large models responsive
Security	Dynamic RLS with USERPRINCIPALNAME()	Restricting data by logged-in user

Remember: a good Power BI analyst spends most of their effort upstream — clean data in Power Query and a solid star-schema model — so that DAX stays simple and visuals just work.